Practical No.: 09

NAME: Bhalerao Tanvi Vitthal
ROLLNO.: 08
COURSE: AI&DS
CLASS: BE
SUB: Computer Laboratory-IV

Title: Perform the data classification algorithm using any Classification algorithm.

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,␣
␣confusion_matrix
```

[2]:
```
# Load iris dataset from seaborn
iris = sns.load_dataset("iris")

# Display first 5 rows
iris.head()
```

[2]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

[3]:
```
# Check dataset information
iris.info()

# Check for missing values
iris.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
```

```
 3   petal_width    150 non-null    float64
 4   species        150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

[3] :
```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

[4] :
```
# Convert species to numeric labels
iris['species'] = iris['species'].astype('category').cat.codes

# Separate features and target
X = iris.drop(columns=['species'])
y = iris['species']

# Display features
print(X.head())

# Display target
print(y.head())
```

```
   sepal_length  sepal_width  petal_length  petal_width
0           5.1          3.5           1.4          0.2
1           4.9          3.0           1.4          0.2
2           4.7          3.2           1.3          0.2
3           4.6          3.1           1.5          0.2
4           5.0          3.6           1.4          0.2
0    0
1    0
2    0
3    0
4    0
Name: species, dtype: int8
```

[5] :
```
# Split data (80% training, 20% testing)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape)
```

```
print("Testing samples:", X_test.shape)
```

```
Training samples: (120, 4)
Testing samples: (30, 4)
```

[6] :

```
# Standardize the dataset

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[7] :

```
# Create Random Forest model

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model
rf_model.fit(X_train, y_train)
```

[7] :

```
RandomForestClassifier(random_state=42)
```

[8] :

```
# Predict test data

y_pred = rf_model.predict(X_test)

print("Predicted Values:")
print(y_pred)
```

```
Predicted Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 2 1 0 2 0]
```

[9] :

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.9
```

[10] :

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.82      0.90      0.86        10
           2       0.89      0.80      0.84        10
```

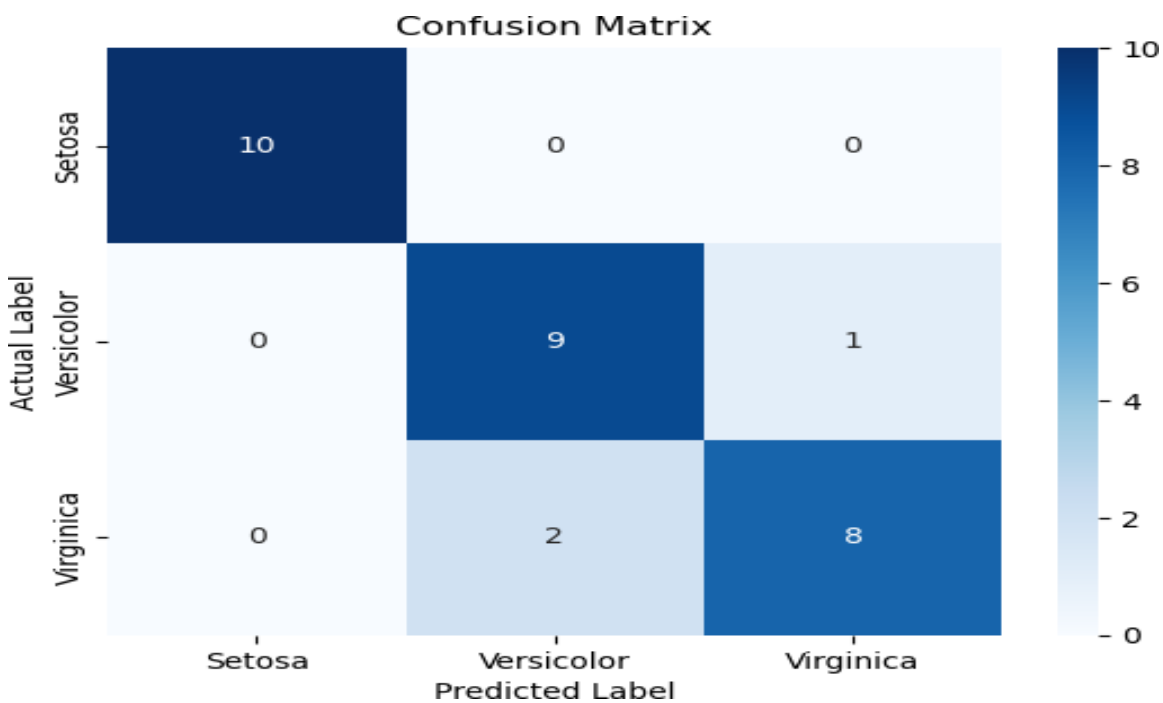
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30

[11] :

```
conf_matrix = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(conf_matrix)
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  2  8
```

[12] :

```
sns.heatmap(
    conf_matrix, annot=True,
    cmap="Blues", fmt="d",
    xticklabels=['Setosa','Versicolor','Virginica'], yticklabels=['Setosa','Versicolor','Virginica']
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix") plt.show()
```